TECH TALK · LLM INFERENCE

LLM Inference
From Pipeline to Testing

Starting from data preparation and the training pipeline, we move into the two stages of inference and the internals of an inference engine, then land on the three categories of testing — performance, accuracy, and stress — plus three load-test cases, and close with where inference is heading.

Pipeline Overview | Inference | Testing | Outlook

CORE CONTENT

1 **LLM Pipeline Overview**
Data Preparation · Pre-training · Fine-tuning · Post-training · Reinforcement Learning

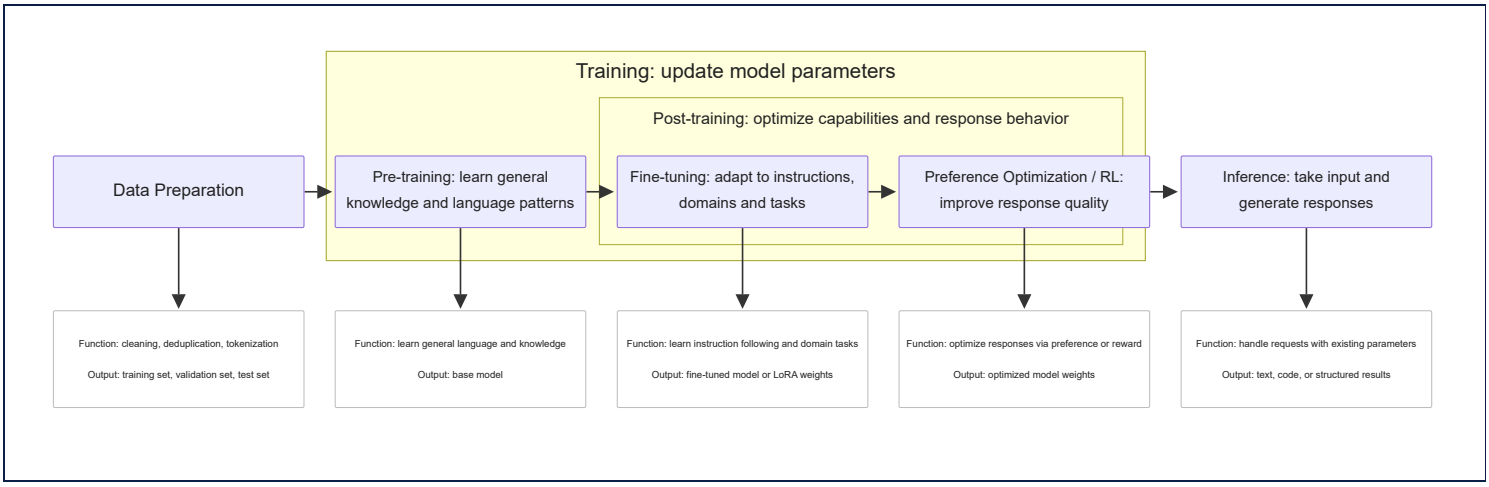
2 **Inference in Depth**
What Is Inference · Two Main Stages · Inference Pipeline · Core Concepts

3 **Inference Testing**
Performance · Accuracy · Stress Testing · Three Load-Test Cases

4 **Inference Trends**
Reasoning Compute · Generation Acceleration · Disaggregation

01 LLM Pipeline Overview

1.1 Overall Pipeline and Concept Relationships



DATA PREPARATION → PRE-TRAINING → FINE-TUNING → PREFERENCE OPTIMIZATION / RL → INFERENCE

1.2 Data Preparation: The Raw Material for Model Learning

Before formal training begins, data has to be collected and processed.

Common sources include web pages, books, papers, code, and Q&A content; multimodal models may also use images, audio, and video.

Data preparation typically involves:

01

Cleaning

Remove garbled text, meaningless content, and low-quality samples.

02

Deduplication

Reduce duplicate data so the model does not over-learn certain content.

03

Filtering

Handle sensitive information, disallowed content, and the like.

04

Tokenization

Convert raw text into Tokens.

05

Dataset Splitting

Separate training, validation, and test sets to reduce benchmark data leakage.

In one sentence:

> “ Model capability depends not only on parameter count, but also on the quality, coverage, and organization of the training data.

1.3 Pre-training: Building General Capability

Pre-training typically starts from an initialized model and uses large-scale data to learn language patterns and knowledge associations.

For a typical autoregressive language model, the core task is:

> CORE TASK
>
> **Given the preceding Tokens, predict the next Token.**

WHAT DOES PRE-TRAINING PRODUCE?

Pre-training typically produces a **base model**.

It may already generate text well, but it cannot necessarily yet reliably:

- Understand and follow user instructions.
- Answer in a specified format.
- Hold a natural multi-turn conversation.
- Handle safety boundaries and inappropriate requests.

Example Given the input “explain what a database is,” a base model may simply continue the sentence rather than give a clear explanation.

1.4 Fine-tuning: Adapting an Existing Model to Specific Needs

Fine-tuning continues training an existing model on new data.

Compared with pre-training from scratch, fine-tuning usually needs less data and compute, so many organizations start from an existing model.

COMMON GOALS INCLUDE:

FINE-TUNING GOAL	EXAMPLE
Follow instructions	Summarize, translate, or generate code on request
Adapt to a domain	Understand financial terminology, process industry documents
Standardize output	Reliably produce JSON with a given structure
Perform specific tasks	Classification, information extraction, test-case generation

1.5 Post-training: Improving Response Behavior and Task Performance

Post-training is a set of optimization steps that follow pre-training; there is no single fixed recipe. It commonly includes:

- Instruction tuning.
- Preference optimization.
- Reinforcement learning.
- Safety behavior training.
- Training for reasoning, tool calling, and similar capabilities.

WHY MIGHT MORE OPTIMIZATION BE NEEDED AFTER SFT?

SFT teaches the model from reference answers, but real responses may still:

- Miss the question or ramble.
- Sound plausible without evidence.
- Ignore user requirements.
- Behave inconsistently on complex tasks.

> THEREFORE
>
> **The model must be told which of several responses are better, and what "better" actually means.**

CONCEPT COMPARISON

CONCEPT	PLAIN EXPLANATION	PRIMARY PURPOSE
Training	The process of adjusting model parameters using data and an optimization algorithm	Give the model new or improved capabilities
Pre-training	Foundational training on large-scale data	Build general language, knowledge, and task ability
Fine-tuning	Continued training on top of an existing model	Adapt to a specific task, domain, or behavior requirement
Post-training	A series of capability and behavior optimizations after pre-training	Improve instruction following, response quality, safety, and reasoning
Inference	Using a trained model to process input and generate output	Deliver model capability to users or business systems

1.6 Reinforcement Learning and Reasoning Ability

The basic idea of reinforcement learning is:

> BASIC IDEA
>
> **The model generates an answer → receives a reward or score → adjusts its parameters from that feedback.**

Rewards can come from human preferences or from verifiable outcomes, for example:

- Whether a math answer is correct.
- Whether generated code passes its tests.
- Whether a tool call accomplishes the goal.
- Whether the output satisfies format constraints.

> It can help the model improve its task strategy and reasoning, but the effect depends on reward design and training method. Evaluation is still required afterward — a rising reward alone does not prove that real capability improved across the board.

02 Inference: Putting a Trained Model to Work

2.1 What Is Inference?

Inference is the process of a model taking an input and generating an output using its existing parameters.

For example:

Input
Summarize this report.
Output
The report covers three main areas...

> KEY DIFFERENCE
>
> **Training typically updates parameters; inference typically only reads them.**

So an ordinary conversation does not automatically teach the model new knowledge and write it into the weights. Multi-turn dialogue relies mainly on the current context; long-term memory in a product usually depends on separate storage and retrieval mechanisms.

2.2 The Two Main Stages of Inference

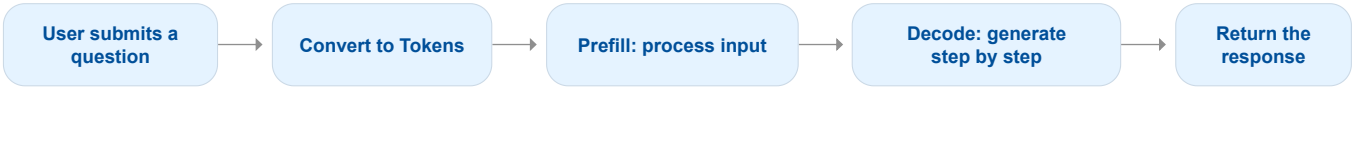
For a typical autoregressive language model, inference splits into:

STAGE	WHAT IT DOES	PRIMARY CONCERN
Prefill	Process the input context and build the state needed for generation	Input length, time to first Token
Decode	Generate output Tokens step by step from the context	Generation speed, per-Token latency

The model typically uses a **KV Cache** to store part of the intermediate results it has already computed, avoiding redundant work in later generation.

> IN ONE LINE
>
> **Prefill processes what the user has already said; Decode generates, step by step, what the model says next.**

2.3 Inference Pipeline



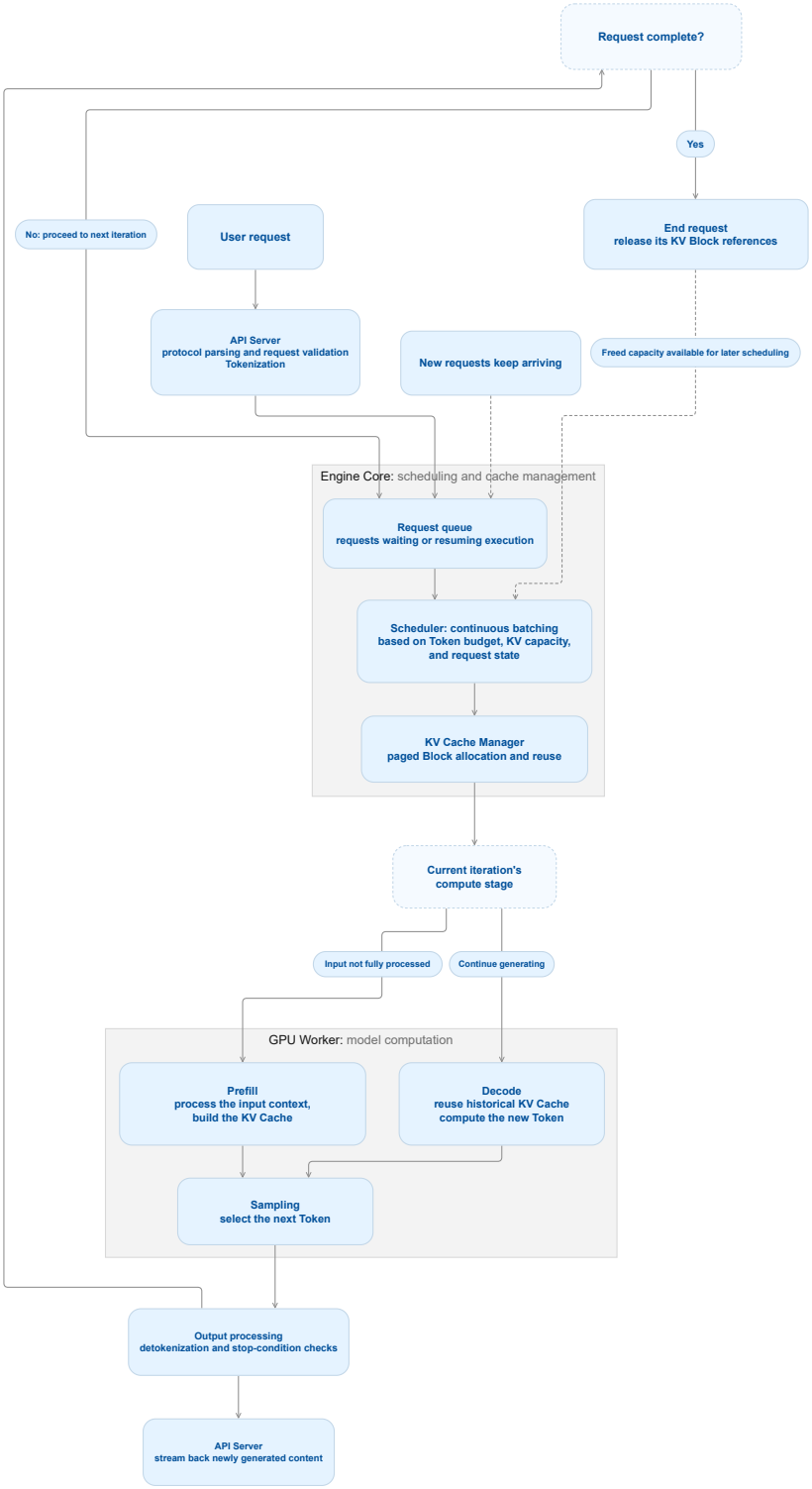
USER SUBMITS A QUESTION → CONVERT TO TOKENS → PREFILL PROCESSES INPUT → DECODE GENERATES STEP BY STEP → RETURN THE RESPONSE

2.4 Core Concepts

CORE CONCEPT	DESCRIPTION	WHY IT MATTERS
Token: the unit of processing	The model splits text into small pieces and processes them one by one; one Token is not necessarily one character	Input length, output length, and processing speed are usually measured in Tokens
Context: the information available for this request	Includes system instructions, the user question, conversation history, and supplementary material	How much the model can reference is limited by the context window
Prefill: processing the input	The model first computes the entire input to prepare for generating a response	The longer the input, the more computation and the longer the wait for the first Token
Decode: generating step by step	The model typically generates one Token at a time, then continues from what it has produced	The longer the output, the longer the whole response takes to finish
KV Cache: caching prior computation	Stores part of the earlier computation so that every new Token does not recompute the whole prefix	Speeds up generation but consumes VRAM
do_sample: sampling	The model produces several candidates	Affects answer diversity and

CORE CONCEPT	DESCRIPTION	WHY IT MATTERS
	with probabilities, then picks one by rule	consistency across repeated requests
Batching: handling multiple requests at once	The inference service groups several user requests so they share compute resources	Raises throughput, but under heavy load it can increase waiting time

2.5 Inference Engine Architecture



ENGINE CORE: SCHEDULING AND CACHE MANAGEMENT · GPU WORKER: MODEL COMPUTATION · API SERVER: STREAMING BACK NEWLY GENERATED CONTENT

03 Inference Testing

3.1 Categories of Inference Testing

CATEGORY	CORE GOAL	IN PLAIN TERMS
Performance testing	Measure speed and resource efficiency under a given load	Under normal use, is it fast?
Accuracy testing	Measure answer quality and task completion	Are the answers right, and do the tasks get done?
Stress testing	Probe capacity limits, overload behavior, and recovery	What happens when there are too many users?

3.2 Performance Testing

ITEM	DETAILS
Method	Hold the model, input length, and output length fixed, then vary concurrency or request rate step by step; compare different hardware, quantization schemes, and inference configurations; cover short input, long input, short output, and long output separately
Tooling	**vLLM bench serve**: measure latency and throughput of an online inference service; **Locust**: simulate custom business requests and load
Metrics	**TTFT**: how long before the answer starts; **TPOT / ITL**: time per Token during generation; **E2E latency**: total request time; **Throughput**: requests or output Tokens per second; **Resource usage**: GPU utilization, VRAM

vLLM's serving benchmark can control request rate and maximum concurrency, and reports Token throughput along with latency metrics.

Example With 1,024 input Tokens and a 256-Token output target, compare latency and throughput at 1, 8, 16, and 32 concurrent requests.

3.3 Accuracy Testing

Here “accuracy” means **answer and task quality**, as distinct from numerical precision such as FP16 or INT8.

ITEM	DETAILS
Method	Use standard datasets and business use cases; compare output against reference answers; run test cases for coding tasks; for open-ended Q&A, combine scoring rubrics, human spot checks, or a model judge
Tooling	**lm-evaluation-harness**: standard language-model benchmarking; **EvalScope**: organizing model evaluation and analyzing results; business scenarios can add custom evaluation scripts
Metrics	**Accuracy**: proportion answered correctly; **EM / F1**: exact match or degree of overlap with the reference; **Pass@k**: how a coding task fares across k candidates; **Task success rate**: whether the business goal was met; **Factual correctness, instruction-following rate**

Both lm-evaluation-harness and EvalScope provide language-model evaluation capabilities; specific tasks require selecting the appropriate evaluation configuration. GitHub

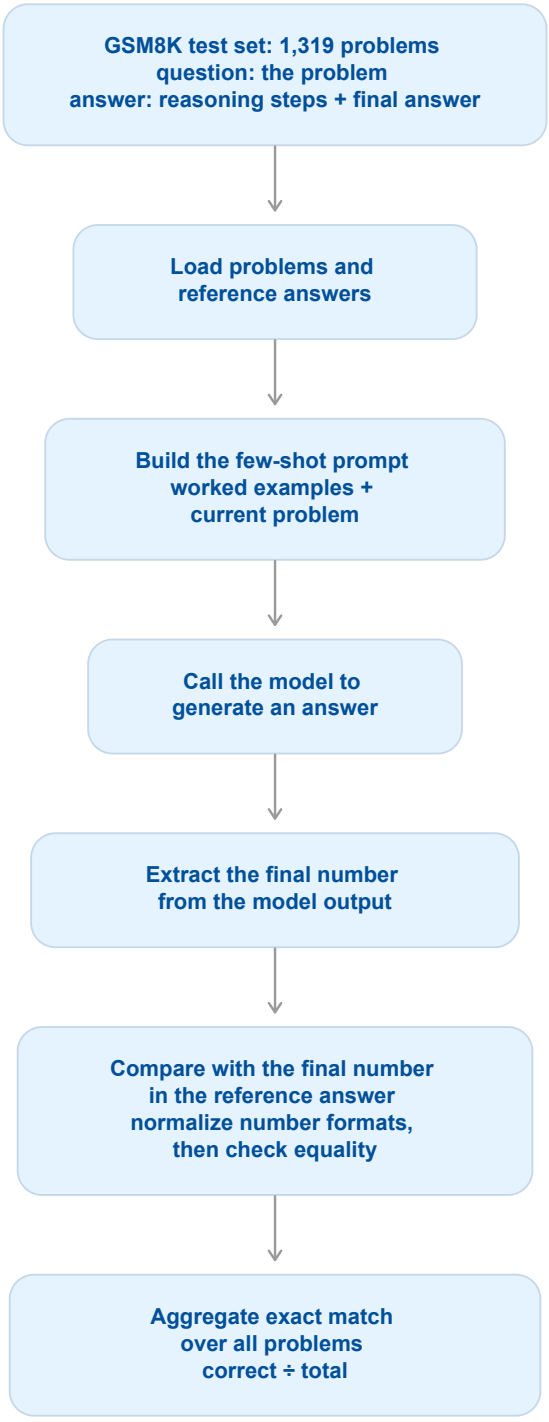
Example Use GSM8K for math problems and HumanEval for code generation, then add internal business Q&A cases.

3.4 Public Datasets

CAPABILITY	COMMON DATASETS	WHAT IT TESTS	COMMON METRICS
General knowledge	MMLU, C-Eval, CMMLU	Multiple-choice questions across many subjects, checking knowledge comprehension and application; the latter two focus on Chinese	Accuracy
Mathematical reasoning	GSM8K, MATH	GSM8K focuses on multi-step grade-school word problems; MATH contains more advanced problems	Final-answer accuracy
Code generation	HumanEval, MBPP	Generate code from a function description or problem statement, then run the test cases	Pass@1, Pass@k
Reading comprehension	SQuAD, multilingual TyDi QA	Read the given passage, then answer related questions	EM: exact-match rate; F1: degree of answer overlap
Instruction following	IFEval	Check whether the response meets verifiable requirements such as a specified format, keywords, or item count	Instruction-level and full-prompt-level pass rates

CAPABILITY	COMMON DATASETS	WHAT IT TESTS	COMMON METRICS
Long-context understanding	LongBench	Long-document Q&A, summarization, multi-document understanding, and more	Task-specific F1, ROUGE, and similar
Factuality and truthfulness	TruthfulQA	Check whether the model readily produces common misconceptions or untrue answers	Truthfulness, informativeness, or multiple-choice score, depending on the evaluation method

3.5 GSM8K Execution Flow



LOAD PROBLEMS → BUILD FEW-SHOT PROMPT → GENERATE ANSWER → EXTRACT NUMBER → COMPARE → AGGREGATE EXACT MATCH

3.6 Shortcomings of Public Datasets

Contamination type	Input contamination: the model has seen the question; Input–answer contamination: the model has seen both question and answer; Near contamination: the question was lightly reworded; Derived contamination: write-ups, solutions, GitHub code, or discussions contain the answer; Synthetic contamination: training data generated by other models reproduces the benchmark.
Detection	String or n-gram matching against known training corpora; Near-duplicate detection such as MinHash or SimHash; Embedding-based semantic similarity;
Mitigation	Generate questions dynamically from recent material; Refresh or rotate the evaluation set regularly; Keep a private held-out test set; Produce structurally varied equivalent forms of public questions; Keep the training set and the frozen evaluation set strictly separate.

3.7 Dataset Auditing

AUDIT DIMENSION	SUB-ITEM	NOTES
Capability coverage audit		First build a target capability tree, then count: samples per capability; coverage per difficulty level; coverage per domain; coverage per language; coverage across input lengths; coverage of each error scenario.
Difficulty, discrimination, and saturation audit	Pass-rate distribution	Look at how many models get each question right. Every model correct: low discriminative value; Every model wrong: check whether it is too hard or simply flawed; Strong models correct, weak models wrong: usually high discriminative value.
	Discrimination	Compute the correlation between item score and overall model capability, or compare pass rates between a high-ability group and a low-ability group.
	Saturation	See how much headroom the current best model has below a perfect score, and whether scores keep climbing quickly toward the ceiling over time.

AUDIT DIMENSION	SUB-ITEM	NOTES
Scoring mechanism audit	Exact Match tasks	Whether numeric equivalence is handled correctly; whether case and whitespace affect scoring; whether fractions and decimals are treated as equivalent; whether option order affects the result; whether correct answers are marked wrong due to a parsing failure.
	Coding tasks	Whether test cases cover enough; whether speculative solutions pass only the visible tests; whether the sandbox is stable; whether timeouts, dependencies, and randomness are applied fairly.
	LLM-as-a-Judge	Agreement with human expert scores; whether it favors longer answers; whether it favors a particular language style; whether swapping answer positions changes the result; whether it favors models from its own family; whether repeated scoring of the same answer is stable; whether the rubric dimensions are genuinely independent.

3.8 Stress Testing

ITEM	DETAILS
Method	Ramp concurrency or request rate in steps until the service is over capacity; generate burst traffic; combine long context and long output to add resource pressure; reduce the load and observe recovery
Tooling	**Locust**: script ramp, burst, and sustained-load scenarios, with distributed load generation; **vLLM bench serve**: measure behavior at different request rates and concurrency limits
Metrics	**Capacity limit**: the maximum load that still meets latency and error-rate targets; **Error rate, timeout rate**; **P95 / P99 latency**; **Queue length, queue time**; **OOM, preemption**; **Recovery time after overload**

CORE DESIGN IDEAS

#	CASE	WHICH LAYER IT HITS	IN ONE LINE
1	Short Prompt, High-Concurrency Decode	VRAM bandwidth / KV Cache management	Very short input + forced long output, 200 concurrent pure Decode
2	Long-Text Prefill Surge	Tensor Core throughput / VRAM capacity	A 2,000-character document + RAG-style question, eating VRAM instantly
3	Mixed Long and Short Sequences	Scheduling fairness / Continuous Batching	Long documents and short Q&A in one pool — see who starves

3.9 Load-Test Cases

CASE 01 Short Prompt, High-Concurrency Decode

CONFIGURATION

case1.json

```
{
  "model": "deepseek_v3",
  "prompt": "<|im_start|>user\nI need to test model performance, please skip other res
  "max_tokens": 100,
  "do_sample": true,
  "top_p": 0.9,
  "top_k": 5000,
  "temperature": 1
}
```

200 concurrent × 10 loops = **2,000 requests**

WHAT IT STRESSES

- **Prefill is extremely short** (< 30 Tokens), so time-to-first-Token is nearly negligible and every request **pours straight into Decode**.
- **Decode is a purely memory-bound task** — every Token generated requires moving all the weights and the entire KV Cache out of HBM again. The GPU's compute units sit largely idle, and **VRAM bandwidth is the real bottleneck**.
- **200 concurrent requests = 200 KV Caches resident in VRAM at once**; the 10 loops make those blocks churn through allocation and release at high frequency, putting PagedAttention's allocator under direct test.
- With `do_sample: true` + `top_k: 5000`: even though the output content is fixed, the engine **cannot take the greedy shortcut** — every step must run the full sort, Softmax, and random sampling, so the sampling kernel's cost is included.

METRICS TO WATCH

METRIC	WHAT TO LOOK FOR
TPOT	The core metric. Whether it degrades **exponentially** at 200 concurrency; the point of degradation is the bandwidth saturation point
TPS	≈ concurrency / average TPOT — shows whether the throughput ceiling has been reached
TTFT	Input is short, but if queueing exceeds 1–2 seconds, the scheduling queue is already congested
VRAM watermark curve	Whether it climbs in steps across the 10 loops and never falls back — a sign of VRAM leak or delayed KV Cache reclamation
Error rate / OOM	Whether 200 KV Caches blow out VRAM

DEFECTS THIS SHOULD CATCH

- **VRAM fragmentation**: after high-frequency allocation and release, a false OOM appears — free memory is visible but no contiguous block can be allocated.
- **Attention breakdown at the output boundary**: forcing "1111..." is an edge-case behavior; some models suddenly emit garbled or hallucinated text after a few dozen Tokens.
- **Truncation failure**: in a degenerate loop, if `max_tokens` does not cut off precisely, a single request can hang.

WHY IT MATTERS This is the **throughput-ceiling yardstick**. Every later tuning change (`max_num_seqs`, `gpu_memory_utilization`) is measured against this TPOT baseline.

3.10 Load-Test Cases (continued)

CASE 02 Long-Text Prefill Surge

CONFIGURATION

case2.json

```
{
  "prompt": "<|im_start|>user\n{a ~2,000-character X-Men Wikipedia article}\nHow did t
  "max_tokens": "${__Random(100,600,)}",
  "temperature": 1,
  "top_p": "${__Random(0,0,)}.${__Random(1,9,)}",
  "top_k": "${__Random(1,500,)}",
  "repetition_penalty": "${__Random(0,2,)}"
}
```

100 concurrent

WHAT IT STRESSES

- **The bottleneck shifts fundamentally**: an input of several thousand Tokens needs one large GEMM to build the initial KV Cache — an unambiguously **compute-bound** task that saturates the Tensor Cores and spikes power draw.
- **A VRAM assassin**: a short text's KV Cache fills gradually during Decode, whereas a long text **grabs a large slab of VRAM the instant the request lands**. At 100 concurrent × several thousand Tokens, tens of GB can vanish in an instant.
- **Simulating a real workload**: this is exactly the concurrency pattern of an enterprise's evening peak for "document accumulation + RAG knowledge-base Q&A."

METRICS TO WATCH

METRIC	WHAT TO LOOK FOR
TTFT	The single most important one. Easily **jumps from a few hundred ms to tens of seconds**
VRAM peak / OOM	The scenario most likely to take the service down outright
HTTP 503 / queue buildup	Watch whether `max_num_seqs` and `max_num_batched_tokens` are being hit
Valid-answer rate	A business-side metric — see “the thinking trap” below

DEFECTS THIS SHOULD CATCH

- **TTFT avalanche**: without **Chunked Prefill**, long requests block one another and jam the scheduling queue.
- **The R1 thinking trap × random truncation**: the answer is stated outright in the source text (“a machine built by Iron Man scattered the Phoenix Force”), making this a simple reading-comprehension question. But DeepSeek-R1 is a reasoning model that leans heavily on `<think>` and will spend several hundred Tokens untangling the X-Men timeline first. Once `max_tokens` randomly lands on a small value, **the request is hard-truncated mid-thought and the user never gets an answer at all** — this is not a performance problem but a **business availability problem**, and the “valid-answer rate” has to be tracked separately.
- **W8A8 quantization × long-text attention**: in long-context attention, W8A8 suffers accuracy loss from clipping activation outliers, and combined with abnormal sampling parameters it readily produces severe hallucination during the `<think>` phase.

WHY IT MATTERS Case 1 stresses **bandwidth**; this one stresses **compute and VRAM capacity**, and it is the first to turn “the model cannot answer” into a quantifiable test result.

3.11 Load-Test Cases (continued)

CASE 03 Mixed Long and Short Sequences

CONFIGURATION

Inject three payloads **into the same concurrency pool**:

ROLE	CONTENT	CONCURRENCY	KEY PARAMETERS
Long sequence	A 2,000+ Token X-Men article, **plain text, no ChatML template**	20	`temperature` random 0.00–1.00
Short sequence	“Please introduce HSBC”	100	`top_k` random 10,000–50,000
Control (canary)	“What are the four major state-owned enterprises?”, standard ChatML	—	Mild parameters, no extremes

case3.json

```
{
  "inputs": "<|im_start|>user\nWhat are the four major state-owned enterprises? <|im_e
  "parameters": {
    "seed": ${__Random(1,184467440)},
    "details": true,
    "do_sample": true,
    "max_new_tokens": ${__Random(100,600,)}
  }
}
```

- **Why it matters:** in a multi-tenant gateway or a mixed-traffic stream, can one abnormal request drag down a perfectly normal request next to it?

WHAT IT STRESSES

- **The physical conflict of mixing lengths**: long-sequence Prefill is **compute-bound** and hogs the compute units; short-sequence Decode is **memory-bound**. Both contend for the same GPU.
- **Kernel-level tail latency**: a `top_k` in the tens of thousands means that for every Token generated, the GPU must **globally sort nearly half the vocabulary**. At a normal `top_k=10–50` it can use register-level partial selection (microseconds); past ten thousand that stops working and it is forced into a Radix/Bitonic global sort.
- **Why the control matters**: it is a probe for the noisy-neighbor effect — will a normal request that logically needs almost no compute be dragged down by a poisonous request beside it?

METRICS TO WATCH

METRIC	WHAT TO LOOK FOR
TTFT degradation from mixing	**How many times slower** a short request's time-to-first-Token is than when run alone — a direct measure of Chunked Prefill's effectiveness
Control-group TPOT inflation	20 ms alone → 200 ms when mixed means **kernel isolation and time-slice scheduling are performing very poorly**
TPS curve	Whether it oscillates sharply in a sawtooth pattern, reflecting Continuous Batching swap-in/swap-out overhead
VRAM fragmentation	Long requests grabbing large blocks and short requests rapidly freeing small ones readily trigger an OOM where memory is free but cannot be allocated

04 Inference Trends

4.1 Inference Trends and Outlook

TREND 01

Inference-time compute becomes a key lever for model capability

By adding thinking steps and generating and verifying multiple candidate answers, a model can perform better on complex tasks such as math and code. Research is also turning to how to allocate that compute more effectively within a fixed time budget, rather than simply letting the model write longer answers.

In plain terms: answer easy questions fast, think harder about hard ones. Testing needs to track accuracy, completion time, and compute cost together.

TREND 02

Generation acceleration moves toward “predict first, then verify”

Speculative decoding lets a faster draft model or another prediction method propose several Tokens first, which the target model then verifies, cutting the cost of Token-by-Token generation. Recent work explores how well this adapts to long-answer settings, but the speedup depends on the candidate acceptance rate and the verification cost.

In plain terms: draft first, then check in bulk. Testing needs to verify the actual speedup and whether output quality holds.

TREND 03

Inference serving moves toward disaggregation and cache reuse

In engineering terms, Prefill and Decode can be deployed separately and optimized independently; routing is starting to assign requests based on KV Cache location and node load, reducing redundant computation. vLLM and NVIDIA Dynamo are already advancing these capabilities.

In plain terms: let each resource do its own job, and reuse what has already been computed. Testing will expand from a single model instance to routing, cache transfer, and the cluster as a whole.

> “ The future of LLM inference will weigh three questions more heavily: how compute can improve answer quality, how generation can be made more efficient, and how serving can scale cooperatively. Inference testing, in turn, needs to evaluate quality, speed, cost, and stability together.

Source: the Obsidian note 大模型推理分享.md

HTML · Single-page scroll layout / engineering-whiteprint